

Industrial Tool

Git & Github

Git Reset :-

Git reset is a powerful command to move your project backward:

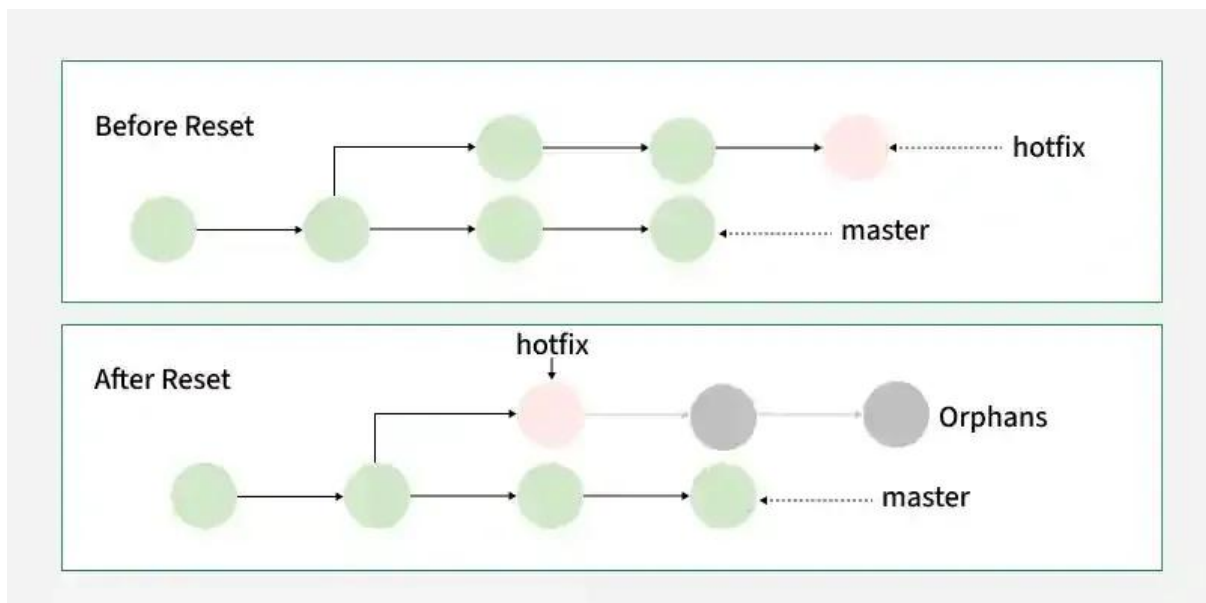
- **HEAD** (which points to the latest commit),
- **index/staging area** (where files are prepared for commit), and
- **working directory** (your actual local files).

The Working of Git Reset :-

Depending on how it's used, git reset allows you to:

- Unstage files that were accidentally added
- Discard unwanted changes in the working directory
- Roll back your current branch to a previous commit

Before and After Reset (Commit History) :-



Before Reset

1. You have two branches:

- **master branch** (green commits)
- **hotfix branch** (pink commit)

2. Both branches have some history (a series of commits).

3. Notice how both branches are pointing to their latest commits.

This refers to the typical state of your working directory., with everything in order.

After Reset

Now imagine you run this command on the master branch:

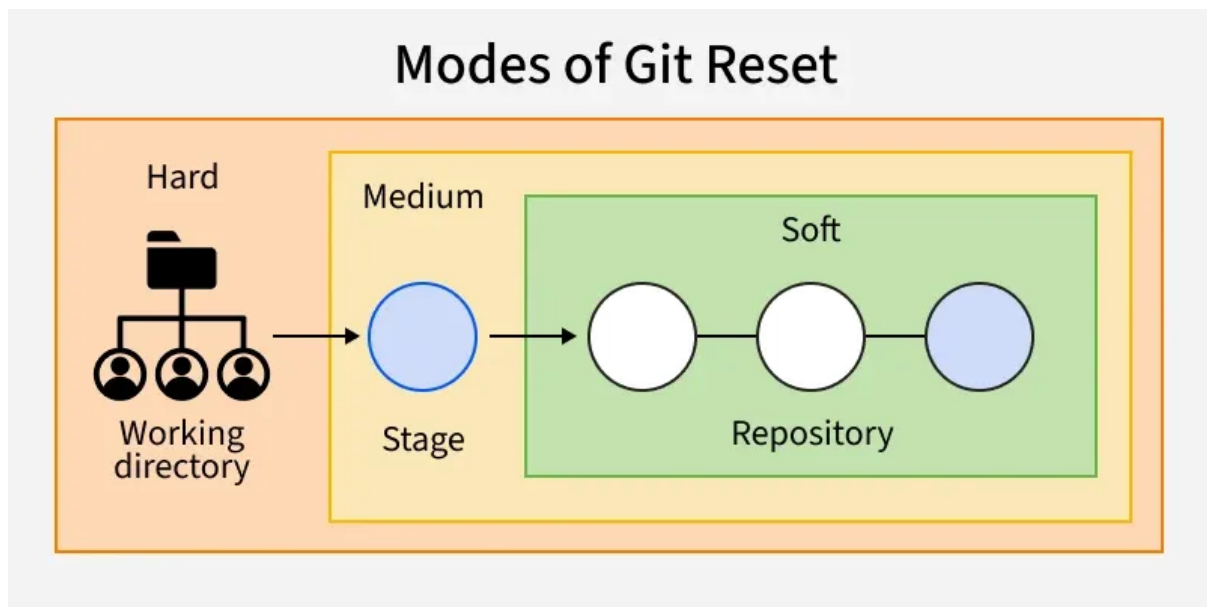
```
git reset --hard <some-older-commit>
```

This **moves the master branch pointer back** to an earlier commit. It effectively reverts your project history to a previous state.

Here's what happens:

- master now points to an older commit
- Later commits become **orphaned** (exist but not shown in logs)
- hotfix branch remains unchanged

Git Reset Modes :-



There are three primary modes of git reset:

1. Soft Reset (--soft)

- Moves the HEAD to an earlier commit.
- Keeps changes staged in the index.
- Leaves your working directory unchanged.

Use case: Undo a commit but keep all changes staged.

2. Mixed(Medium) Reset (--mixed) (default)

- Moves HEAD and updates the index to match the target commit.
- Unstages changes.
- Leaves the working directory unchanged.

Use case: You added files with git add but want to unstage them.

3. Hard Reset (--hard)

- Moves HEAD, index, and the working directory to the specified commit.
- All changes (staged + unstaged) are lost.

Use with caution: It is irreversible and will delete your changes permanently.

Example :-

1. Unstage Files Without Losing Changes

To unstage changes you have added with git add, but keep them in your working directory:

git reset

This is a mixed reset and is helpful when you want to modify what's staged without affecting the actual file content.

2. Revert to a Previous Commit Without Losing Work

To move the HEAD to a previous commit and keep your changes staged:

git reset --soft <commit-hash>

This lets you rewrite history and prepare your changes for a cleaner commit sequence.

3. Completely Discard All Changes

To discard all changes and reset your branch to a specific commit:

git reset --hard <commit-hash>

This will erase everything both staged and unstaged changes and set your branch exactly to the specified commit. Be extremely careful, as this action is irreversible and can result in data loss.

Git Remote :-

- A Git remote refers to a repository hosted on GitHub, GitLab, or a server.
- It connects your local repository to a remote repository.
- It enables sharing and synchronization of code.
 1. Push your local changes to the remote repository.
 2. Pull updates from the remote repository to your local one.

Git Origin :-

- origin is the default name for the remote repository URL.
- It points to where your code is stored (GitHub/GitLab).
- It acts like a shortcut for the remote location.
 1. origin is the key (the name),
 2. and the remote URL is the value (the actual address).

`git push origin main`

This tells Git to push your local main branch to the remote repository named origin.

Managing Git Remote Repositories :-

Git remote commands help you connect, manage, and interact with remote repositories like GitHub or GitLab. Here are the commonly used commands

1. Add a Remote Repository

To establish a connection between your local repository and a remote repository, use the following command:

`git remote add origin <URL>`

```
nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\GFG0569\Pictures\Screenshots\codinggfg> git remote add origin https://github.com/pranjal-dennis-nanda/ticket-maker-with-local-storage-.git
>>
PS C:\Users\GFG0569\Pictures\Screenshots\codinggfg> git remote -v
origin https://github.com/pranjal-dennis-nanda/ticket-maker-with-local-storage-.git (fetch)
origin https://github.com/pranjal-dennis-nanda/ticket-maker-with-local-storage-.git (push)
```

Add a Remote Repository

2. View Remote Repositories

To display a list of linked remote repositories along with their URLs for the local system user attempting to connect to a GitHub server.

git remote -v

```
PS C:\Users\GFG0569\Pictures\Screenshots\codinggfg> git remote -v
origin https://github.com/pranjal-dennis-nanda/ticket-maker-with-local-storage-.git (fetch)
origin https://github.com/pranjal-dennis-nanda/ticket-maker-with-local-storage-.git (push)
```

View Remote Repositories

3. Rename a Remote

To change the name of an existing remote that you are currently connected to on the server.

git remote rename <oldname> <newname>

4. Remove a Remote

To unlink a remote repository from your local repo use the following command. If the repository name does not exist you can write origin instead of the repository name.

git remote remove <repositoryname>

```
PS C:\Users\GFG0569\Pictures\Screenshots\codinggfg> git remote remove hello
PS C:\Users\GFG0569\Pictures\Screenshots\codinggfg> git remote -v
PS C:\Users\GFG0569\Pictures\Screenshots\codinggfg>
```

Remove a Remote

5. Show Remote Details

To display information about a specific remote, including branches and fetch/push URLs use the following command.

git remote show <name>

```
PS C:\Users\GFG0569\Pictures\Screenshots\codinggfg> git remote show origin
* remote origin
Fetch URL: https://github.com/pranjal-dennis-nanda/ticket-maker-with-local-storage-.git
Push URL: https://github.com/pranjal-dennis-nanda/ticket-maker-with-local-storage-.git
HEAD branch: main
Remote branch:
  main new (next fetch will store in remotes/origin)
```

Show Remote Details

Important Git Remote Commands :-

Below are some of the most useful Git remote commands that developers commonly use in real projects:

Remote Setup and Management :-

Command	Description
git remote add <name> <url>	Adds a new remote repository (e.g., origin)
git remote -v	Shows all remotes and their fetch/push URLs
git remote rename <old> <new>	Renames an existing remote
git remote remove <name>	Removes a remote from the local repository
git remote show <name>	Shows detailed info about a remote (branches, URLs, etc.)
git remote set-url <name> <new-url>	Updates the URL of an existing remote

Working with Remotes :-

Command	Description
git clone <url>	Clones a remote repository to your local machine
git fetch <remote>	Fetches changes from the remote without merging
git pull <remote> <branch>	Fetches and merges changes from a remote branch

Command	Description
git push <remote> <branch>	Pushes your local branch to a remote branch
git push --set-upstream origin <branch>	Sets upstream for tracking and pushes branch to remote

Remote Branch Handling :-

Command	Description
git branch -r	Lists all remote-tracking branches
git checkout --track origin/<branch>	Creates a local branch tracking a remote branch
git push origin --delete <branch>	Deletes a branch from the remote repository
git remote prune origin	Removes references to deleted branches from the remote

GitPush With (Visual Studio) VS Code :-

- Visual Studio Code (VS Code) is a powerful and versatile code editor widely used by developers.
- One of its standout features is its seamless integration with Git.
- This makes version control simple and efficient.
- In this article, we'll explore how to push code to Git using VS Code.

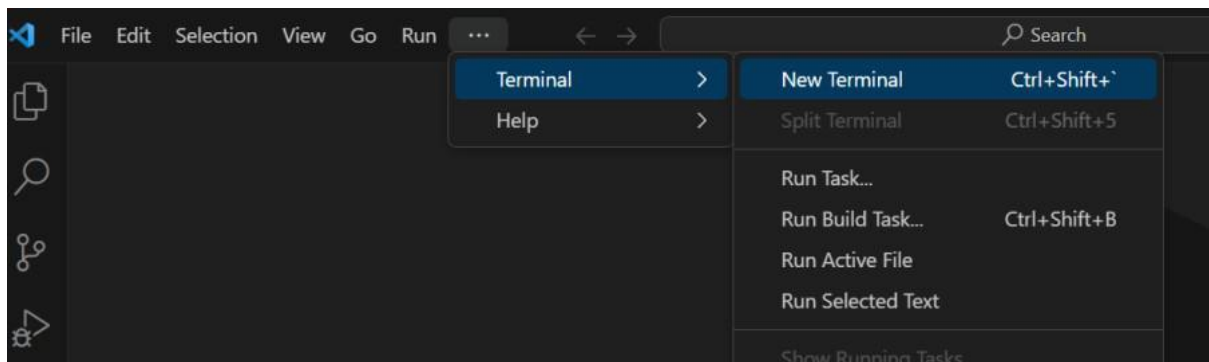
Steps To Create And Add Git Repository in VS Code :-

Step 1: Open the VS code, then open a new terminal and create a new empty folder using this command

```
mkdir gfg
```

Go to this folder using the following command.

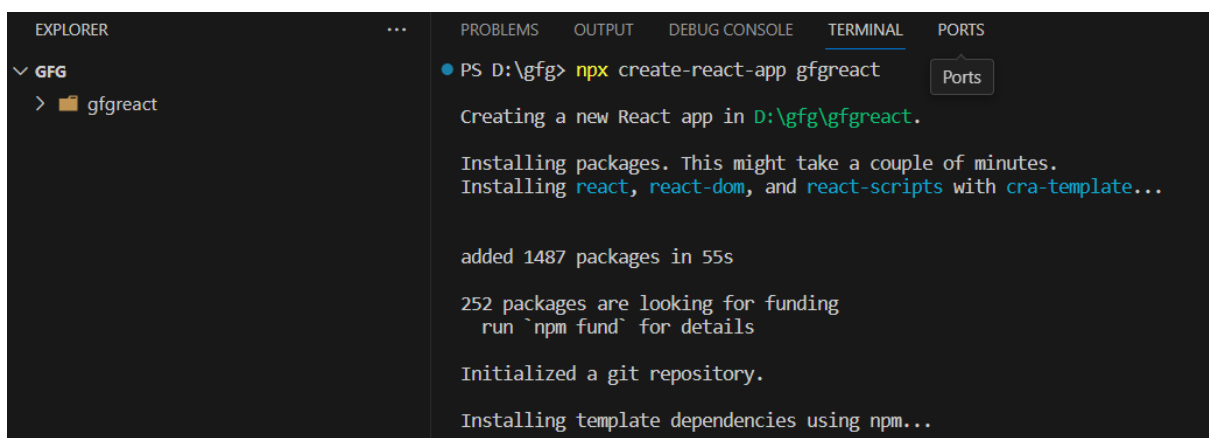
```
cd gfg
```



open terminal

Step 2: Then open our folder new terminal and create new react project using this command

```
npx create-react-app gfgreact
```



create react project

Step 3: Then initialize with git repository using this command

git init

```
● PS D:\gfg\gfgreact> git init
  Reinitialized existing Git repository in D:/gfg/gfgreact/.git/
○ PS D:\gfg\gfgreact> █
```

git init

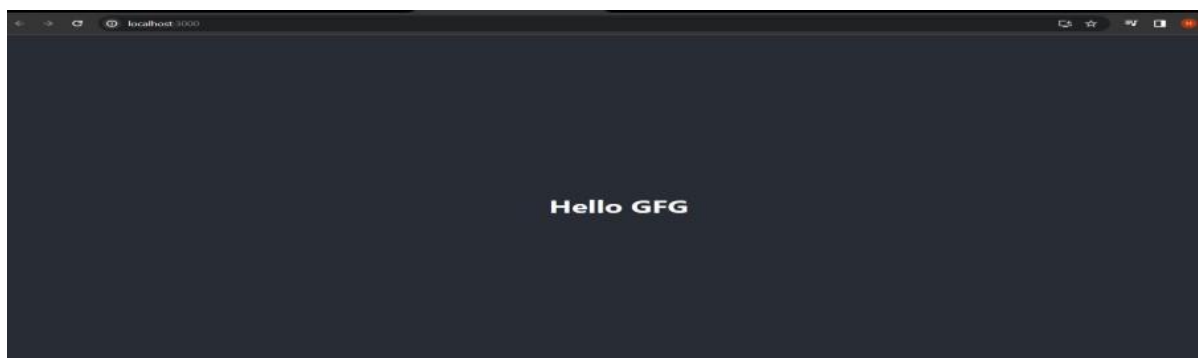
Step 4: Go to the react project and open src folder and open App.js file and add some code then

run this project using this command: npm start

```
import './App.css';
```

```
function App() {
  return (
    <div className="App">
      <header className="App-header">
        <h2>Hello GFG</h2>
      </header>
    </div>
  );
}
```

```
export default App;
```



run

Step 5: Return the vs code terminal and check the status of git repository using this command

git status

```
● PS D:\gfg\gfgreact> git status
● On branch master
  Changes not staged for commit:
    (use "git add <file>..." to update what will be committed)
    (use "git restore <file>..." to discard changes in working directory)
        modified:   src/App.js

  no changes added to commit (use "git add" and/or "git commit -a")
PS D:\gfg\gfgreact> █
```

git Status

Step 6: Then add all changes in git using this command:

git add

Check the status using this command

git status

```
● PS D:\gfg\gfgreact> git add .
warning: in the working copy of 'src/App.js', LF will be replaced by CRLF the next time Git touches it
○ PS D:\gfg\gfgreact> █
```

git add .

```
● PS D:\gfg\gfgreact> git status
○ On branch master
  Changes to be committed:
    (use "git restore --staged <file>..." to unstage)
        modified:   src/App.js

PS D:\gfg\gfgreact> █
```

git status

Step 7: Then commit all changes using this command : git commit -m "change the app.js file" and check the status using this command

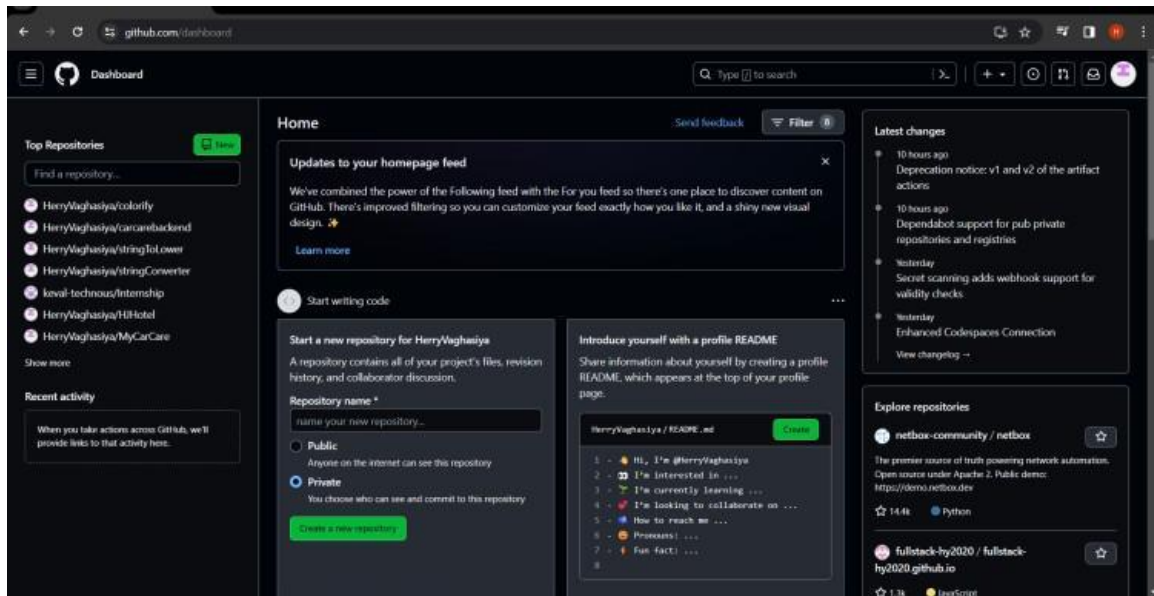
git status

```
● PS D:\gfg\gfgreact> git commit -m "change the app.js file"
[master 35d90cf] change the app.js file
 1 file changed, 1 insertion(+), 12 deletions(-)
● PS D:\gfg\gfgreact> git status
On branch master
nothing to commit, working tree clean
○ PS D:\gfg\gfgreact> █
```

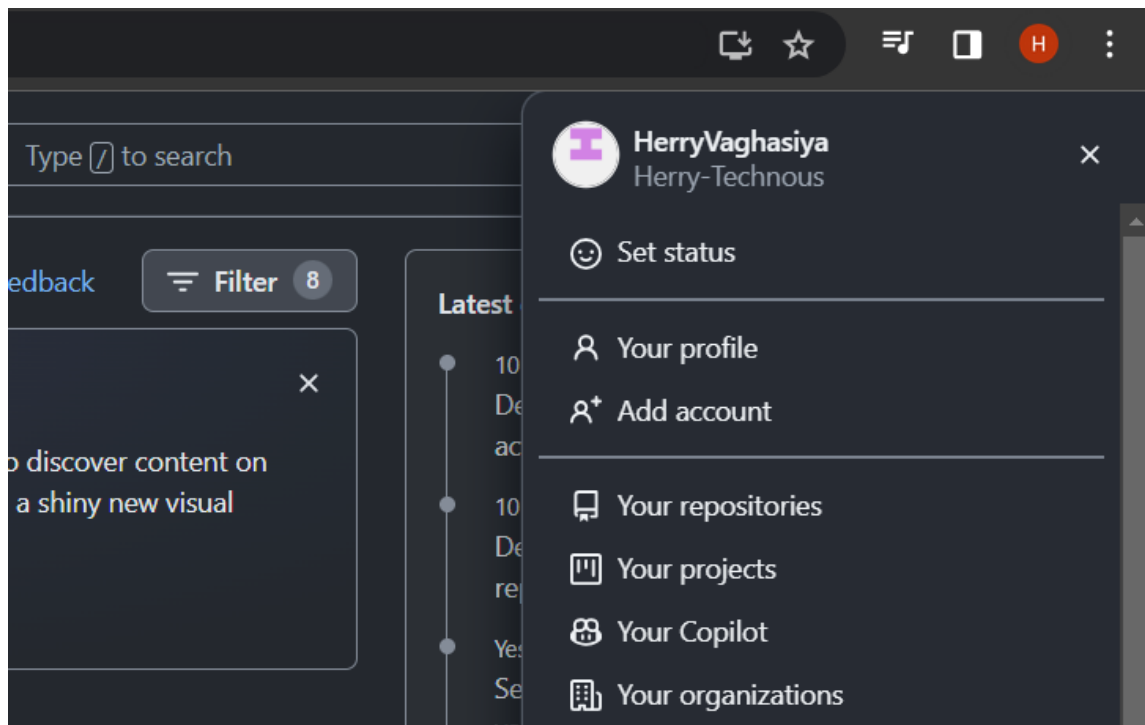
commit all changes

Step 8: These are the changes made in the local system but we want to upload the project which we created in your system to GitHub then follow the below steps to upload :

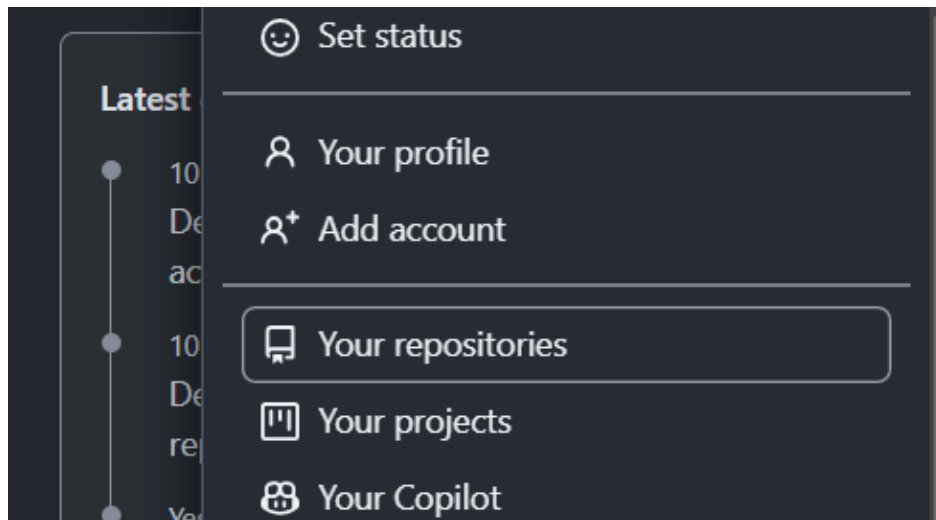
1. Go to the GitHub
2. then click your profile
3. select your repositories folder



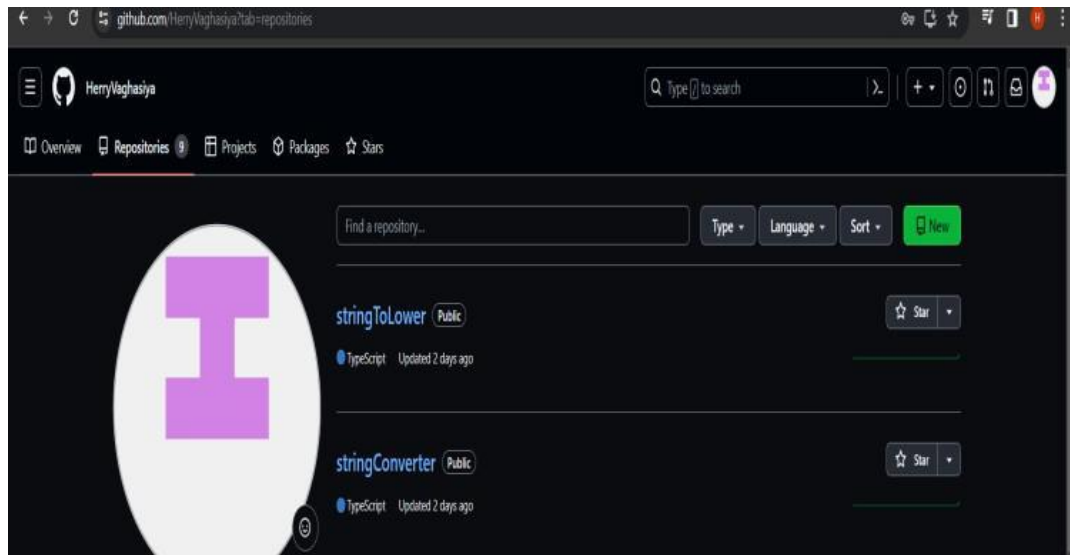
Go to GitHub



profile

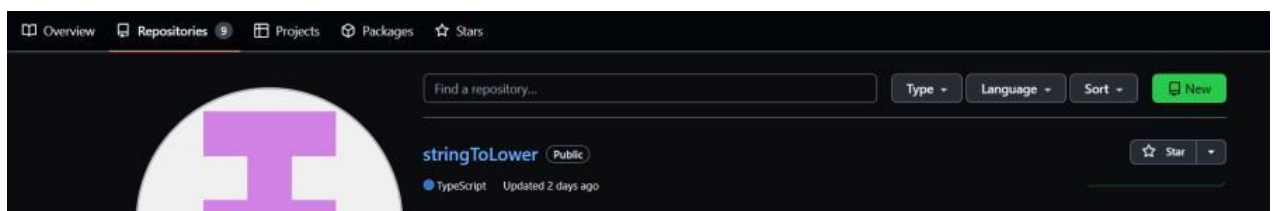


select option



go to repository

Step 9: Then click on new button and create new repository and give the name like GFGRepo and check that our repository is created successfully or not then copy the repository link.



new button

 New repository

Type  to search

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk (*).

Owner *

 HerryVaghasiya

Repository name *

GFGRepo

✔ GFGRepo is available.

Great repository names are short and memorable. Need inspiration? How about [super-octo-engine](#)?

Description (optional)

☒  **Public**
Anyone on the internet can see this repository. You choose who can commit.

repo name

☒  **Public**
Anyone on the internet can see this repository. You choose who can commit.

☐  **Private**
You choose who can see and commit to this repository.

Initialize this repository with:

☐ **Add a README file**
This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

.gitignore template: None ▾

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license

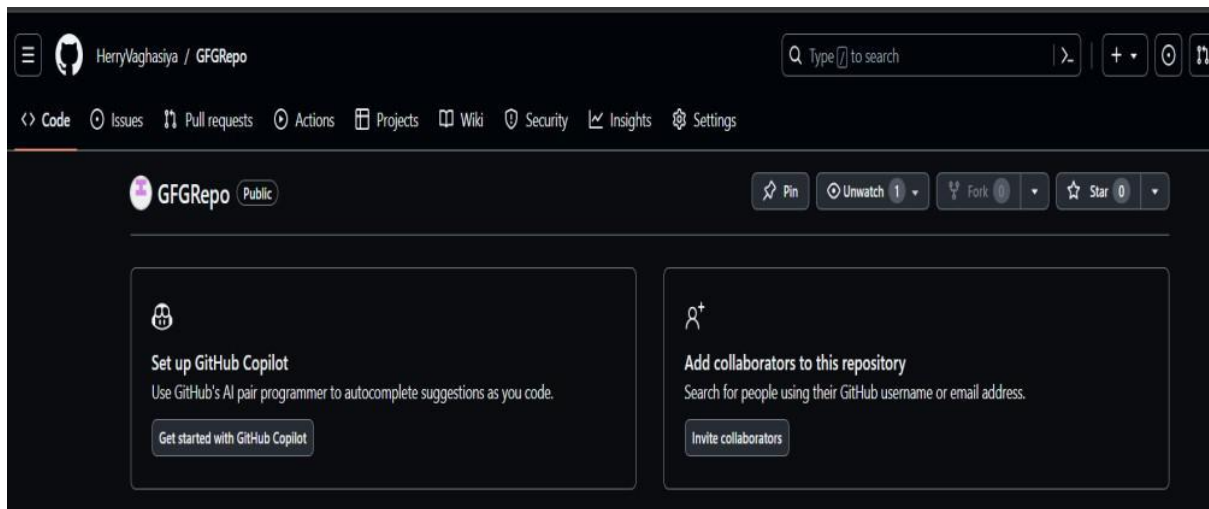
License: None ▾

A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

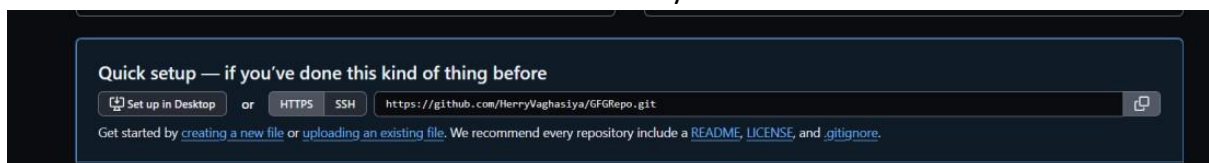
 You are creating a public repository in your personal account.

Create repository

click this button



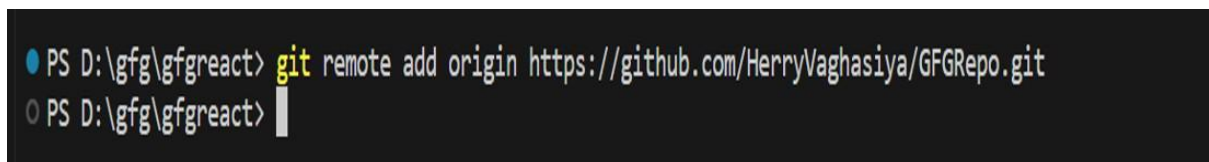
successfully



copy this link

Step 10: Then go to vs code terminal and get the remote of our git repository using this command

git remote add origin https://github.com/HerryVaghasiya/GFGRepo



get remote

Step 11: check that repository remote is success fully added or not using this command

git remote -v

Check our git repository branch using this command

git branch

Change the branch name master to main because GitHub change its default branch name master to main so rename our branch name using this command:

git branch -M main

Check branch name

git branch

```
● PS D:\gfg\gfgreact> git remote -v
origin https://github.com/HerryVaghasiya/GFGRRepo.git (fetch)
origin https://github.com/HerryVaghasiya/GFGRRepo.git (push)
○ PS D:\gfg\gfgreact>
```

check remote

```
● PS D:\gfg\gfgreact> git branch
* master
● PS D:\gfg\gfgreact>
```

check branch

```
● PS D:\gfg\gfgreact> git branch -M main
● PS D:\gfg\gfgreact> git branch
* main
○ PS D:\gfg\gfgreact>
```

change and check branch name

Step 12: Then push our code in GitHub repository using this command

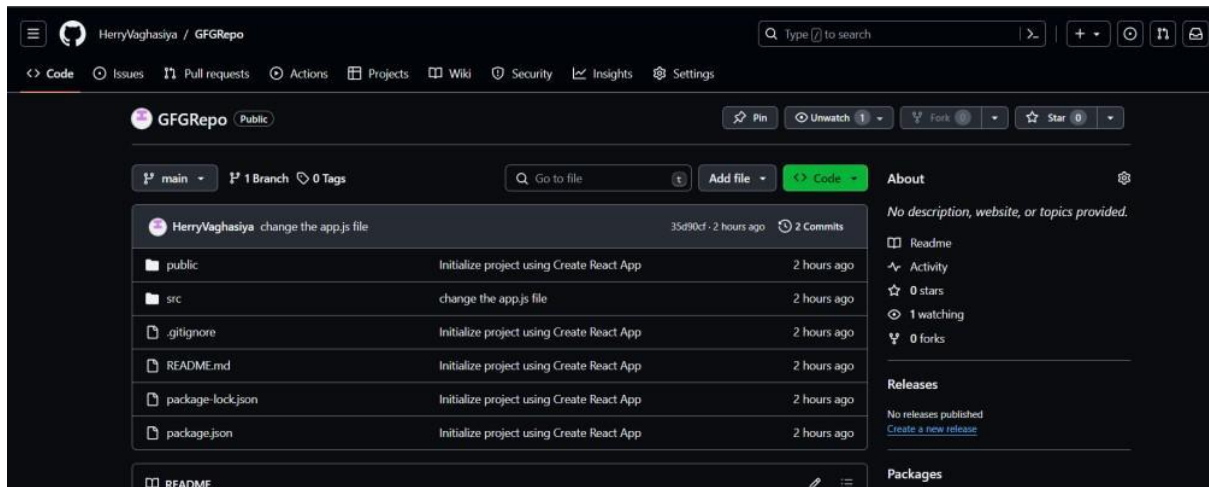
git push -u origin main

(Note : -u means set upstream. If you want to work on the same project for a long time, there is a shortcut key that will set the branch name as default so that you don't have to give the same branch name every time.)

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS D:\gfg\gfgreact> git push -u origin main
Enumerating objects: 26, done.
Counting objects: 100% (26/26), done.
Delta compression using up to 16 threads
Compressing objects: 100% (26/26), done.
Writing objects: 100% (26/26), 180.00 KiB | 7.83 MiB/s, done.
Total 26 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), done.
To https://github.com/HerryVaghasiya/GFGRRepo.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
○ PS D:\gfg\gfgreact>
```

push the code

Step 13: Then Go to GitHub repository and refresh the page and check that our project is successfully pushed or not.



project successfully pushed